AIOps PLE Platform

Architecture Design Document

PLE + adaTT-Based Financial Product Recommendation System
Technical Design Document (Internal Reference)

Version 1.0 — 2026-04-01

Design vs Implementation Note

This document describes the **design intent and target architecture**. The current implementation may be a subset of the design, and some components described herein may not yet be implemented. Refer to the codebase and `pipeline_state.json` for implementation status.

Table of Contents

1. System Overview

1.1. System Purpose

AIOps PLE Platform is an end-to-end multi-task learning platform for **financial product recommendation**. It targets banks, card companies, and financial holding companies, generating personalized recommendations for comprehensive financial products—deposits, cards, loans, investments—along with **explainable rationale** simultaneously.

The final deliverable of AI recommendation is not a probability (0.73) but a **reason the customer can accept**. The persuasion target is always a person, and three levels of explanation are required:

Audience	Question	Expected Level
Customer	"Why this product?"	Trust -> Conversion
Banker	"Why recommend this to this customer?"	Sales rationale
Regulator	"Why was this decision made?"	Regulatory compliance (EU AI Act, FSS Guidelines)

1.2. Target Users and Operating Environment

- **Financial ML Operations Staff**: Realistically 1–2 people. N models x explanation modules x ensemble logic = N management points. A single config system must enable unified management.
- **Hardware**: 1–4 GPU scale. Since MLP parameter scaling is infeasible, structural inductive bias is used to secure expressiveness.
- **Infrastructure**: On-Prem (Airflow + DuckDB) or AWS (SageMaker + S3). The architectural philosophy remains identical; only the infrastructure layer changes.

1.3. On-Prem vs AWS Comparison

Aspect	On-Prem (Current)	AWS (Target)	Migration Rationale
Orchestration	Airflow 86 DAGs ($300/mo)	Step Functions 5 ($0/mo)	Pay-per-execution
Training	Local GPU	SageMaker Spot (70% savings)	Parallel ablation
Storage	DuckDB files	S3 Parquet	Durability, IAM
Serving	FastAPI + Docker	Lambda / ECS Fargate	Auto-switch by scale
Experiment Mgmt	MLflow (Docker)	SageMaker Experiments	Eliminate server maintenance cost
Monitoring	Custom drift_monitor	SageMaker Model Monitor	Managed service

2. Design Philosophy

Four core design principles serve as the starting point for all technical decisions.

2.1. Principle 1: Inherent Explainability

The system pursues **structural explanation** rather than post-hoc explanation (SHAP/LIME).

- **SHAP/LIME limitations**: Separated from the model, so explanations diverge from internal behavior. Explanations change drastically with slight input changes. Separate computation per inference multiplies serving latency.
- **Structural approach**: Explanations are derived from the model structure itself (gate, evidential, contrastive). A single forward pass produces both inference and explanation simultaneously.

The Heterogeneous Expert architecture makes this possible:

- CGC gate weights themselves serve as explanations: "Temporal(0.35), DeepFM(0.28), HGCN(0.22)"
- Expert names directly map to business context: "Time-series pattern contributed 35%, feature interaction 28%, product hierarchy 22%"
- With homogeneous MLP experts, "MLP 3 contributed" carries no business meaning

2.2. Principle 2: Graceful Degradation

Even if one expert becomes useless, the remaining experts naturally compensate through gate redistribution.

- Ablation proves "removing any expert does not cause catastrophic performance degradation"
- Financial sector characteristics: Unlike big tech's aggressive "AUC +0.01 = N billion in revenue" experimentation, financial firms face "regulatory sanctions for model malfunction" -> conservative operations, stability first

2.3. Principle 3: Extensibility

Adding new features/tasks requires only config changes. adaTT automatically learns new relationships without modifying existing structures.

- **Pool/Basket/Runtime 3-tier**: Code(Pool) -> Config(Basket) -> Training(Runtime) separation
- Zero code changes for domain switching: simply replace config files from `configs/financial/` to `configs/ecommerce/`

2.4. Principle 4: Manageability

The entire pipeline (feature generation -> training -> distillation -> serving -> recommendation rationale) is managed through a unified config system (`pipeline.yaml` + `feature_groups.yaml`).

- **Config-Driven**: A single YAML defines data/tasks/model/infrastructure. New problems without code changes.
- **Registry Pattern**: Expert, Task, Feature, Model, Tower are all registered via plug-in pattern.
- **Schema-First**: The data schema determines the entire pipeline.

3. Architecture Decision History

3.1. From ALS to Black-Litterman

The existing financial product recommendation system was based on ALS (Alternating Least Squares) collaborative filtering. When the decision to adopt MLOps was made, next-generation model evaluation began.

Initial candidates:
1. DL model family (DeepFM, Wide&Deep, AutoInt)
2. GBM model family (XGBoost, LightGBM, CatBoost)
3. DL + GBM ensemble

During ensemble evaluation, there was an attempt to apply the financial portfolio optimization **Black-Litterman model** to recommendations. The idea was to treat each model's prediction as an "expert view" and integrate them via Bayesian updating weighted by uncertainty (risk).

3.2. Black-Litterman Drop Rationale

Critical limitations were identified at the design stage:

Limitation	**Details**
Business interpretation impossible	Each model's contribution is blended during Bayesian updating -> "Why was this product recommended?" becomes hard to explain. Fatal in the regulatory environment.
Structural mismatch	Fundamental difference between financial portfolios (continuous weight allocation) and product recommendation (discrete selection)
View matrix automation difficulty	Uncertainty estimation for each model is subjective and hard to automate
Multi-task limitation	Insufficient means to express inter-task relationships when integrating churn/recommendation/segment/value into a single BL framework

3.3. PLE + adaTT Selection Process

The question was reframed: "How to blend multiple models well?" (ensemble) -> "Is there a structure where multiple experts collaborate within a single model?" (MoE/PLE).

MTL (Multi-Task Learning) family evaluation:

Architecture	**Characteristics**	**Limitation**
Shared-Bottom	Single expert shared across all tasks	Severe negative transfer
MMoE	N experts + per-task gate	All experts exposed to all tasks -> insufficient specialization
PLE	Shared + task-specific expert separation, CGC gate	Selected

Why PLE was selected:
- Expert network = internal ensemble (per-expert specialization)
- CGC gate = per-task expert weighting -> explainability
- Single model training/deployment -> 1 management point, fixed serving cost
- What Black-Litterman tried externally (uncertainty-based integration of expert opinions) is solved internally, data-driven, within the model structure

3.4. Decision Flow Summary

```
ALS (existing)
  | "MLOps adoption, next-gen model needed"
DL + GBM ensemble evaluation
  | "Simple ensemble insufficient for multi-task integration"
Black-Litterman attempt
  | "Financial portfolio != product recommendation, dropped at design stage"
PLE + adaTT selected
  | "Prototype validated on-premises"
AWS migration
  | "Parallel ablation + end-to-end serving needed"
Benchmark data + Ablation execution
```

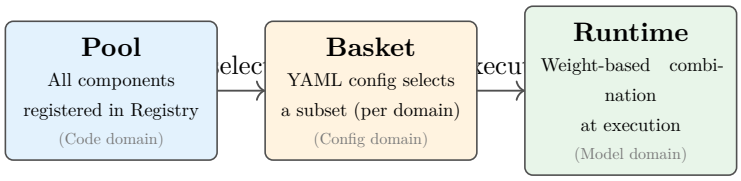

Figure 1: Pool / Basket / Runtime 3-tier architecture.

```
  | (planned)
Distillation + Lambda serving
```

4. PLE + Heterogeneous Expert Basket

4.1. Limitations of Original PLE and Introduction of Heterogeneous Experts

The original PLE (Tencent, 2020) shared experts consist of **homogeneous MLP x N** (same structure, different initialization). Expressiveness increases only by adding parameters, so lightweight models lack representational power.

Our approach: shared expert = **7 heterogeneous experts**. Each expert processes the same data from a different perspective with a different inductive bias, and the CGC gate learns "which perspective is useful for this task." Lighter than a single large MLP yet more expressive.

4.2. Pool / Basket / Runtime 3-Tier

Tier	Role	Change Agent
Pool	Register all available components in Registry	Developer (code addition)
Basket	Select subset for a specific pipeline via YAML	Operator (config swap)
Runtime	Weighted combination of selected components during execution	Model (automatic during training)

4.3. Expert Pool — 11 Registered Types

#	Registry Name	Class	Primary Axis	Basket
1	`deepfm`	DeepFM Expert	State (feature interaction)	O
2	`temporal_ensemble`	Temporal Ensemble	Timeseries (Mamba+LNN+Transformer)	O
3	`hgcn`	Unified HGCN	Hierarchy (hyperbolic graph)	O
4	`perslay`	PersLay Expert	Snapshot (TDA global)	O
5	`causal`	Causal Expert	Snapshot (NOTEARS DAG)	O
6	`lightgcn`	LightGCN Expert	Item (graph collaborative filtering)	O
7	`optimal_transport`	OT Expert	Snapshot (Sinkhorn)	O
8	`mlp`	MLP Expert	State (basic)	—
9	`mamba`	Mamba Expert	Timeseries (SSM)	—
10	`autoint`	AutoInt Expert	State (Self-Attention)	—
11	`xdeepfm`	XDeepFM Expert	State (CIN + Deep)	—

4.4. 7-Expert Selection Rationale

Each expert possesses a non-overlapping inductive bias:

Expert	Inductive Bias	Pattern Captured with Fewer Parameters
DeepFM	Feature interaction	2nd-order cross effects between features

Expert	Inductive Bias	Pattern Captured with Fewer Parameters
Temporal Ensemble	Composite time series (Mamba+LNN+Transformer)	Long/short-term/discontinuous time-series integration
HGCN	Hierarchical structure (hyperbolic space)	Product category tree
PersLay	Topological structure	TDA persistence shapes
LightGCN	Graph relationships	Customer-product collaborative filtering
Causal	Causal relationships	Causal direction (DAG constraint)
Optimal Transport	Distribution matching	Distribution differences across customer segments

4.4.1. Temporal Ensemble: Intra-Expert Ensemble

Financial time series have compound structures that no single model can capture. The Temporal Expert itself is designed as an internal ensemble of 3 models:

Model	Strength	Weakness
Mamba (SSM)	Long-range dependency, O(n) efficiency	Weak on irregular intervals
LNN (Liquid NN)	Adapts to discontinuous/irregular data	Poor at long-context summarization
Transformer	Short-context extraction	$O(n^2)$, inefficient for long sequences

4.5. CGC Layer + Attention

`CGCLayer` is the core building block of PLE. For each task, a gating network combines shared + task-specific expert outputs.

- **dim_normalize=True**: When expert output dimensions differ, `sqrt(mean_dim/dim)` scaling corrects dimensional imbalance
- **bias_high/bias_low**: Injects initial bias toward domain-relevant experts
- **entropy regularization**: Prevents expert collapse

Stacked PLE: 3 CGC layers. Layer 0 uses the heterogeneous Expert Basket + FeatureRouter (each expert receives a different input_dim subset), while Layers 1–2 use homogeneous MLP experts for abstraction. Because per-expert input dims are heterogeneous (27D–168D), `dim_normalize` corrects dimensional imbalance in the CGC gate.

4.6. Dual-Registry Architecture

```
Expert Pool Registry (core.model.experts.registry.ExpertRegistry)
    +-- AbstractExpert(input_dim, config)
    +-- 11 types registered

Expert PLE Registry (core.model.ple.experts.ExpertRegistry)
    +-- BaseExpert(input_dim, output_dim, dropout)
    +-- For CGCLayer default expert creation

Expert Basket (core.model.ple.experts.ExpertBasket)
    +-- Pool Registry -> Basket selection -> Inject into CGCLayer.shared_experts
```

5. adaTT + Task Group

5.1. 4 Financial DNA Groups

The initial design considered per-GMM-cluster task sub-heads, but with K=20, T=12, this would produce 240 sub-heads -> unmanageable, high overfitting risk. The direction was shifted to **grouping tasks by financial DNA perspective**:

Group	Financial DNA	Included Tasks	intra	inter
engagement	Does the customer respond?	next_mcc, top_mcc_shift	0.8	0.3
lifecycle	Where is the customer?	churn_signal, product_stability	0.7	0.3
value	How valuable is the customer?	mcc_diversity_trend	0.6	0.3
consumption	What will the customer buy?	nba_primary, cross_sell_count, will_acquire_* (5)	0.7	0.3

A total of **12 tasks** organized into 4 semantic groups.

5.2. Adaptive Task Transfer (adaTT)

- **Intra-group**: Strong transfer between tasks in the same group (0.6–0.8)
- **Inter-group**: Weak transfer between different groups (0.3) — minimizing interference
- **Negative transfer threshold**: Automatically blocks transfer when performance degrades
- **EMA decay**: Stabilizes transfer weights
- **Warmup/freeze epochs**: Initial stabilization

5.3. Loss-Level Transfer: Logit Transfer (3-Method Dispatch)

Explicit causal relationships between tasks are modeled through 3 edges:

Source	Target	Method	Meaning
has_nba	nba_primary	output_concat	Subscription status -> Which product
churn_signal	product_stability	output_concat	Churn -> Product stability
next_mcc	nba_primary	hidden_concat	Next merchant category -> Next product (feature sharing)

Three transfer methods:

- **residual**: source output -> Linear -> tower_dim, residual addition
- **output_concat**: source output concatenated with tower input -> Linear -> tower_dim
- **hidden_concat**: source pre-tower hidden concatenated with tower input -> Linear -> tower_dim

`logit_transfer_strength: 0.5` — transfer ratio.

5.4. HMM Triple-Mode Projection

Three HMM modes are routed to task groups:

Task Group	HMM Mode	Meaning
engagement	behavior	Behavior mode -> Response/activity
lifecycle	lifecycle	Lifecycle mode -> Churn/retention
value	journey	Journey mode -> Value/spending
consumption	journey	Journey mode -> Consumption pattern

Per-mode 16D -> `task_expert_output_dim` projection, followed by additive fusion into tower input.

5.5. Multidisciplinary Per-Task Routing

24D multidisciplinary features are routed as 6D slices to 4 task groups:

Task Group	Discipline	Dimension
engagement	chemical_kinetics	[0:6]
lifecycle	epidemic_diffusion	[6:12]
value	crime_pattern	[12:18]
consumption	interference	[18:24]

6. Feature Pipeline

6.1. 5-Axis Feature Classification

All features are classified along 5 axes, and each axis is mapped to a corresponding Feature Generator and Expert.

Axis	Temporal Dependency	Rate of Change	Representative Data	Processing Method
State	None (static)	Nearly invariant	Age, gender, enrollment date	Feature interaction (FM)
Snapshot	Long-term (monthly/quarterly)	Slow	12-month transaction topology, HMM states	Long-term pattern extraction
Timeseries	Short-term (daily/weekly)	Fast	Recent 90-day transaction sequence	Sequence modeling (SSM)
Hierarchy	None (structural)	Slow	MCC code hierarchy, product categories	Hyperbolic embedding
Item	None (relational)	Medium	Customer-product interactions	Graph collaborative filtering

6.2. 17 Feature Groups (1205D Input + 6 Passthrough = 1211D)

The raw input tensor is 1205D (17 feature groups) plus 6 passthrough columns. With FeatureRouter active, each expert receives a designated subset of this tensor. Per-expert input dims: deepfm=977D, temporal_ensemble=116D, hgcn=58D, perslay=32D, causal=129D, lightgcn=955D, optimal_transport=95D (mlp=57D is the per-task tower in PLE CGC, not a shared expert). Feature group-to-expert routing is group-level, auto-built at `build_model()` time from `target_experts` declarations in `feature_groups.yaml` — no hard-coded column routing. Model parameter count: V1 baseline (349-feature schema) measured at 2.70M; V2 (1211-feature schema with lag tensor 800D, rolling stats 20D, multihot 24+31D) parameter count pending re-measurement (the lag tensor 800D routes to deepfm and lightgcn, expected to add 400-450K to first-layer projections).

6.2.1. 4 Base Groups (transform type)

Generated by transforming existing raw columns. No Generator required.

- `base_rfm`: RFM-based features (quantile transform)
- `base_demographics`: Demographic features
- `base_product`: Product holding status
- `base_activity`: Activity metrics

6.2.2. 13 Generated Groups (generate type)

Dedicated Feature Generators produce features.

Group	Generator	Output D	Description
tda_global	tda_global	16D	Population-level persistent homology (H0+H1, 365d window)
tda_local	tda_local	16D	Individual-level persistent homology (H0+H1, 90d window)
hmm_states	hmm	25D	Journey/lifecycle/behavior state estimation (triple-mode)
mamba_temporal	mamba	50D	Selective SSM on transaction sequences (GPU-precomputed)
product_hierarchy	graph	34D	Poincaré embedding of 24 products in 6 L1 groups
merchant_hierarchy	merchant_hierarchy	27D	MCC L1→L2 tree Poincaré embeddings (HGCN input)

Group	Generator	Output D	Description
graph_collaborative	graph	66D	Customer-product bipartite LightGCN embedding
gmm_clustering	gmm	22D	Soft posterior probabilities (20 clusters)
model_derived	model_features	27D	KMeans(5) + Bandit/MAB(4) + LNN(18)
txn_lag_tensor	lag_extractor	800D	Right-aligned lag flattening: K=200 × 4 seq features
txn_rolling_stats	rolling_stats_extractor	20D	4 windows (7/30/90/180d) × 5 metrics
nba_label_multihot	topn_multihot_extractor	24D	Fixed-vocab 24-product multi-hot (NBA recommendation set)
mcc_top30_multihot	topn_multihot_extractor	31D	Top-30 MCC multi-hot + others column

6.3. 11 Academic Disciplines (Multidisciplinary Feature Design)

Discipline	Adopted Element	Meaning for Financial Customers
Topology	TDA Persistent Homology	Structural shape of consumption patterns
Hyperbolic Geometry	Hyperbolic GCN	Distortion-free embedding of product hierarchy
Stochastic Processes	HMM State Transitions	Customer lifecycle stage tracking
Control Theory	Mamba (State Space Model)	Long-range behavioral dependency
Economics	Permanent/transitory income, marginal utility	Structural decomposition of spending capacity
Financial Engineering	Risk metrics, Bandit explore/exploit	Product exploration propensity
Graph Theory	LightGCN	Behavioral transfer among similar customer groups
Statistics	GMM Clustering	Soft segmentation
Causal Inference	Causal DAG (NOTEARS)	Causal direction between behaviors
Optimal Transport	Sinkhorn Optimal Transport	Distribution shift across segments
Neural ODE	Liquid Neural Network	Adaptation to irregular time intervals

Key insight: These perspectives are non-overlapping. Topology's "shape" and economics' "utility" are entirely different aspects of the same customer, each contributing independently (proven by ablation).

6.4. Dual Role of Features: Training + Recommendation Rationale

The value of multidisciplinary features is not just training performance (AUC contribution). Even if removing TDA features only drops AUC by 0.01, the explanation "Your consumption pattern has been consistently showing a stable shape" is impossible to generate without TDA.

Business context reverse mapping is built for all features:

- `hmm_lifecycle_prob_growing` -> "Growth-stage customer"
- `mamba_temporal_d3` -> "Spending increase trend over the last 3 months"
- `hgcn_hierarchy_d5` -> "Position close to investment product category"

6.5. 3-Stage Normalization Pipeline

```
Stage 1: Power-law detection (skew+kurt -> log-log R^2) + log1p copy generation
Stage 2: StandardScaler (continuous columns only, binary excluded, TRAIN fit only)
Stage 3: Power-law _log copies are not scaled (raw magnitude preserved)
```

- Scaler is **fit on TRAIN split only**. val/test are transform-only with the train-fitted scaler.
- Binary columns are excluded from scaling.

7. Training Pipeline

7.1. Phase 0: Data Preparation

A 10+ stage PipelineRunner transforms raw data into training-ready tensors.

Stage	Name	Description	GPU
1	DataAdapter	DuckDB-native raw data load, AdapterRegistry-based	—
1.5	TemporalPrep	Sequence truncation (drop last month), prod_* recalculation	—
2	SchemaClassifier	Classify all columns into 5-Axis	—
3	EncryptionPipeline	PII -> SHA256 -> INT32 (domain-specific salt)	—
4	FeatureGroupPipeline	Run 8 Generators + PowerLawAwareScaler	cuDF
5	LabelDeriver	Config-driven 12-label generation	—
5.5	LeakageValidator	Sequence/correlation/product/temporal 4-way leakage check	—
6	SequenceBuilder	flat -> 3D tensor (event_seq, session_seq)	—

Phase 0 runs on **CPU instances**. GPU instances are not wasted on Phase 0.

7.1.1. 4-Layer Data Leakage Prevention

1. **Sequence truncation**: 17 months -> 16 months (drop last month)
2. **Product recalculation**: Recalculate prod_* based on month 16 state
3. **Temporal split**: temporal split + gap_days (minimum 7 days)
4. **LeakageValidator**: Sequence/correlation/product/temporal verification, auto-remove features with >0.95 correlation

7.1.2. Data Processing Backend Policy

Priority: cuDF (GPU) -> DuckDB (CPU columnar) -> pandas (last resort fallback, only for fewer than 10K records). Direct use of `pd.read_parquet()`, `pd.concat()`, `df.apply()` and similar pandas calls is avoided.

7.2. Phase 1–3: Ablation Study (23 Scenarios, 48 Total Configurations)

Systematic experiments across 4 dimensions x multiple scenarios:

Phase	Dimension	Content	Job Count
1	Feature Group	full + base_only + bottom-up + top-down	16
2	Expert	deepfm baseline + bottom-up + top-down + mlp_only	16
3	Task x Structure	4 tiers x 4 structures (shared_bottom / ple_only / adatt_only / full)	16

- **Bottom-up**: base + X -> measures independent contribution
- **Top-down**: full – X -> measures irreplaceability

7.3. Model Training (Common to Phase 1–3)

7.3.1. Per-Task Loss Dispatch

Loss Type	Module	Usage	Task Type
focal	FocalLoss(alpha, gamma)	Imbalanced binary classification (calibrated alpha)	binary
huber	SmoothL1Loss	Outlier-robust regression	regression
mse	MSELoss	Basic regression	regression
ce	CrossEntropyLoss(weight)	Multi-class (auto class_weights)	multiclass
infonce	InfoNCELoss(temperature)	Contrastive learning	contrastive

In AMP (Mixed Precision) environments, tower output is cast to FP32 before loss computation (overflow prevention).

7.3.2. Uncertainty Weighting (Kendall et al.)

$$L_{\text{total}} = \sum_k \left[\exp(-s_k) \cdot L_k + \frac{s_k}{2} \right]$$

s_k: learnable log-variance for task k. Tasks with higher uncertainty -> automatic weight reduction.

7.3.3. Evidential Deep Learning (Config-Gated)

Task Type	Distribution	Parameters	Uncertainty
Binary	Beta(alpha, beta)	alpha, beta	1/(alpha+beta)
Multiclass	Dirichlet(alpha)	alpha_1..K	K/sum(alpha)
Regression	Normal-Inverse-Gamma	mu, v, alpha, beta	beta/(v*(alpha-1))

7.3.4. SAE Regularization (Detached, Config-Gated)

Anthropic-style Sparse Autoencoder. Applied to the shared expert concatenated output. **Detached** so it does not affect main model gradients (analysis-only sidecar). Tied weights, pre-bias centering, ReLU activation.

8. Distillation + Serving

8.1. Phase 4: Knowledge Distillation (PLE -> LGBM)

```
PLE Teacher (GPU training)
    | Soft label generation (temperature=5.0)
    | Store in S3
LGBM Student (CPU training)
    | alpha=0.3 (30% hard + 70% soft)
    | LGBM gain importance feature selection (top-k features)
    | Save lightweight model
Serving (real-time: LGBM ~5ms, batch: PLE)
```

- **Temperature**: 5.0 — maximizes information content of soft labels
- **Alpha**: 0.3 — 30% hard label + 70% soft label blend
- **Fidelity validation**: Teacher-student prediction agreement verification

8.2. Phase 5: Serverless Serving (Lambda)

8.2.1. Serving Architecture

- **Real-time**: LGBM student -> Lambda (5ms latency)
- **Batch**: PLE teacher -> SageMaker Batch Transform
- **Scale switching**: Lambda (default) ECS Fargate (large scale) automatic switching

8.2.2. 3-Layer Fallback Architecture (FallbackRouter)

Distillation failures on minority-class tasks motivated a layered serving design. The `FallbackRouter` monitors per-task-type fidelity on a rolling window and escalates when any layer falls below its configured threshold:

Layer	Model	When Active	Latency
L1	LGBM student (distilled)	Default: fidelity > threshold on all task types	~5ms
L2a	PLE teacher (direct inference)	LGBM fidelity drops on binary or multiclass tasks	~80ms
L2b	Bedrock LLM (Sonnet)	Regulatory explanation required or confidence < floor	~800ms
L3	Rule-based fallback	All models unavailable or circuit breaker open	<1ms

Calibration is maintained separately per layer. L2b (Bedrock) handles the 3-agent reason pipeline (FactExtractor → TemplateEngine → SelfChecker) and is invoked only when explainability depth exceeds what the student's IG attribution can provide. End-to-end serving cost including rationale generation is approximately $0.69 per cycle.

8.2.3. FD-TVS Composite Scoring

Task-level predictions are integrated into a single recommendation score:

Task	Weight
nba_primary	0.30
cross_sell_count	0.15

Task	Weight
churn_signal	0.15
product_stability	0.10

8.2.4. DNA Modifier

Segment-level weight adjustment: TOP(1.3), PARTICULARES(1.0), UNIVERSITARIO(0.8), UNKNOWN(0.7).

8.2.5. Constraint Engine

Constraint	Description
Fatigue	Maximum 5 messages within 7 days
Eligibility	min_score > 0.05, max_churn_prob < 0.6
Owned Product	Exclude already-owned products via prod_* prefix
Product Tier	Minimum tenure: standard 3 months, growth 6 months, premium 12 months
Top-K Diversity	Ensure diversity via MMR (lambda=0.5)

8.2.6. Recommendation Rationale Generation Pipeline

```
[Model Inference] -> [IG Attribution: Top Features] -> [Business Reverse Mapping]
-> [LLM Agent: Context Composition -> Natural Language Recommendation Rationale]
```

Four levels of explanation are provided simultaneously:

1. **gate weight**: "Which perspective contributed" (expert-level)
2. **contrastive**: "Why this and not that" (contrastive)
3. **evidential**: "How confident is this" (uncertainty quantification)
4. **SAE**: "Which internal concepts were activated" (neuron-level)

9. Monitoring + Compliance

9.1. Pipeline Tracking

Artifact	Location	Purpose
`pipeline_manifest.json`	output_dir/	Full pipeline config snapshot
`pipeline_state.json`	output_dir/	Per-stage completion/failure status, resume support
`feature_stats.json`	output_dir/	Zero-variance, NaN ratio, feature column count
`label_stats.json`	output_dir/	Class balance, positive rate

9.2. Per-Stage Checkpoints

Stage	Artifact	Format
Feature	`features.parquet`	Parquet
Label	`labels.parquet`	Parquet
Sequence	`sequences.npy`, `seq_lengths.npy`	NumPy
Scaler	`scaler_params.json`	JSON
Leakage	`leakage_report.json`	JSON

9.3. Audit Artifacts

```
audit/
+-- schema/          <- Schema validation results
+-- encryption/      <- PII processing audit log
+-- scaler/          <- scaler_params.json
+-- labels/          <- label_transforms.json
+-- leakage/         <- LeakageValidator results
+-- fidelity/        <- Distillation fidelity verification
```

9.4. Interpretability Pipeline (Stage 8.5)

Analysis	Purpose	Output
Integrated Gradients (IG)	Feature attribution measurement	attribution scores
Expert Redundancy CCA	Inter-expert redundancy detection	CCA correlation matrix
CGC Gate Analysis	Per-task expert weight analysis	attention heatmap
HGCN Interpretable	Hierarchical structure explanation	hierarchy paths
Multi Interpreter	Multidisciplinary interpretation integration	structured reasons
Template Reason Engine	Natural language recommendation rationale	text templates
XAI Quality Evaluator	Explanation quality evaluation	quality scores
Model Card	Auto-generated model documentation	model_card.json

9.5. Encryption Pipeline (Stage 3)

```
Schema (pii: true)
    | derive_from_schema()
EncryptionPolicy (per source, per column)
    |
EncryptionPipeline.process_source()
    +-- Step 1: Drop (phone, email, SSN, etc.)
    +-- Step 2: SHA256 Hash (domain-specific salt)
    +-- Step 3: Integer Index (hash -> INT32)
    +-- Step 4: Audit report
```

16 PII domains defined (CUSTOMER, ACCOUNT, CARD, MERCHANT, etc.). SaltManager manages per-domain salts from AWS Secrets Manager or local storage.

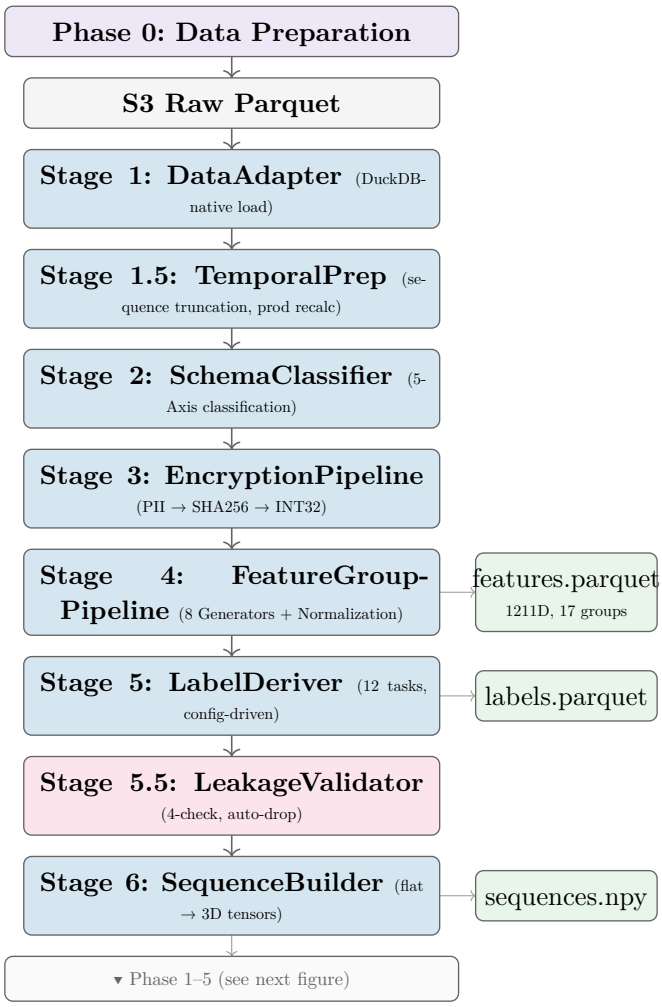

Figure 2: End-to-End pipeline (1/2): Phase 0 data preparation — from raw S3 ingestion to feature/label/sequence tensor storage.

10. End-to-End Data Flow

10.1. Full Data Flow Diagram

10.2. Internal Model Data Flow

FeatureRouter is **active**: each expert receives only its designated feature group subset, not the full 1205D tensor (1211D total (17 groups)). Per-expert input dims: deepfm=977D, temporal_ensemble=116D, hgcn=58D, perslay=32D, causal=129D, lightgcn=955D, optimal_transport=95D (mlp=57D is the per-task tower in PLE CGC, not a shared expert). Routing is group-level, auto-built from `target_experts` in `feature_groups.yaml`. Model parameters 2.8M post-FeatureRouter pruning (varies with group toggle; lag tensor 800D routes to deepfm and lightgcn).

10.3. GPU/CPU Acceleration Mapping

Stage	Target	CPU Path	GPU Path
1	Data loading	DuckDB (primary)	cuDF (optional)
4	Generator execution	pandas fallback	cuDF primary (cuML for GMM)
4	TDA persistence	ripser (NumPy)	cuPY + ripser (5–10x)
4	StandardScaler	NumPy	cuPY (3–5x on 100M+ rows)
7	Training data loading	PyArrow (zero-copy)	—
8	Model training	PyTorch CPU	PyTorch CUDA + AMP

GPU acceleration is optional; CPU paths are used as automatic fallback when cuDF/cuPY is not installed.

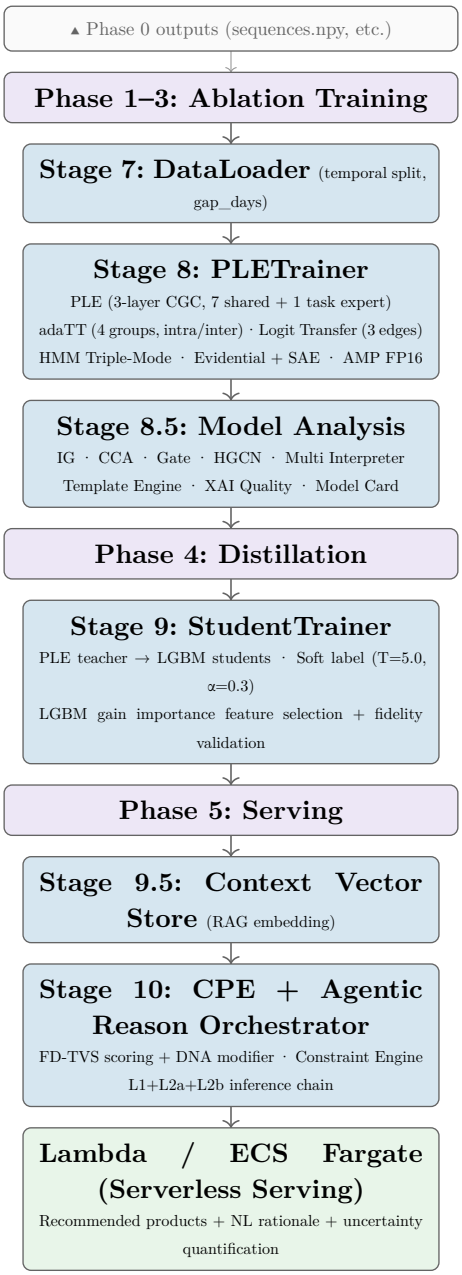

Figure 3: End-to-End pipeline (2/2): Phase 1–3 training → Phase 4 distillation → Phase 5 serving.

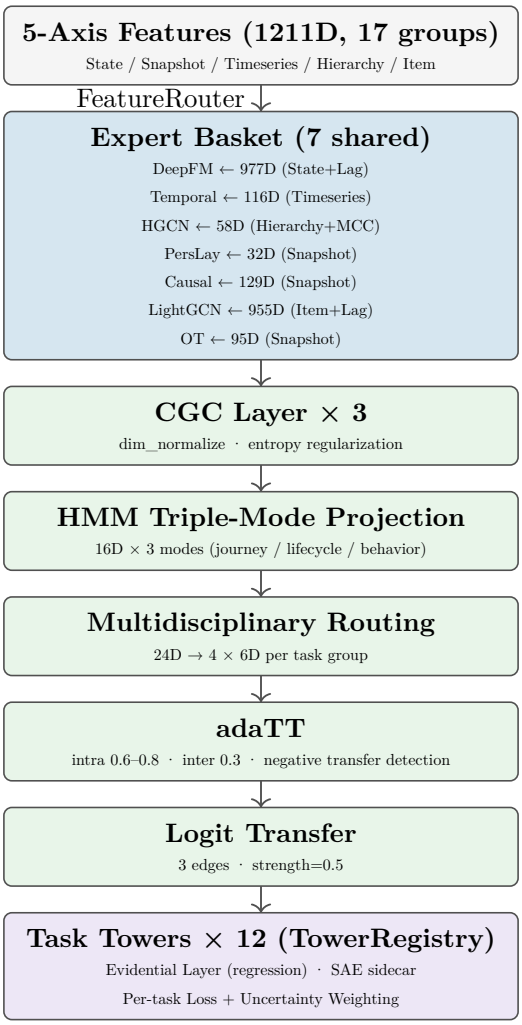

Figure 4: Internal model data flow: FeatureRouter slices per-expert subsets from the 1205D feature tensor (1211D, 17 groups). Routing is group-level, auto-built from `target_experts` in feature_groups.yaml. deepfm=977D, temporal=116D, hgcn=58D, perslay=32D, causal=129D, lightgcn=955D, ot=95D.

11. Appendix: PLEModel Build Automation

When `PLEModel.__init__(config: PLEConfig)` is called, the following are built automatically in order:

1. `_build_extraction_layers()` — Stacked CGC + FeatureRouter
2. `_build_cgc_attention()` — Per-task attention (dim_normalize)
3. `_build_task_experts()` — GroupTaskExpertBasket or MLP fallback
4. `_build_hmm_projectors()` — 3 modes x projection
5. `_build_adatt()` — Adaptive Task Transfer
6. `_build_logit_transfer()` — 3-method dispatch, 3 edges
7. `_build_multidisciplinary_routing()` — 24D -> 4 x 6D
8. `_build_task_towers()` — TowerRegistry (standard/contrastive)
9. `_build_evidential_layers()` — NIG for regression (config-gated)
10. `_build_sae()` — Sparse Autoencoder (config-gated)
11. `_build_task_loss_fns()` — `build_loss()` per task
12. `_build_loss_weighting()` — Uncertainty / GradNorm / DWA / Fixed

11.0.1. PLEInput Data Container

```
@dataclass
class PLEInput:
    features: Tensor                    # (batch, input_dim)
    feature_group_ranges: Optional[Dict] # group->(start,end) for routing
    cluster_ids: Optional[Tensor]        # (batch,) cluster assignment
    cluster_probs: Optional[Tensor]      # (batch, n_clusters) soft probs
    targets: Optional[Dict[str, Tensor]] # {task_name: label}
    hyperbolic_features: Optional[Tensor]   # (batch, 20)
    tda_features: Optional[Tensor]          # (batch, 70)
    collaborative_features: Optional[Tensor]# (batch, 64)
    hmm_journey: Optional[Tensor]           # (batch, 16)
    hmm_lifecycle: Optional[Tensor]         # (batch, 16)
    hmm_behavior: Optional[Tensor]          # (batch, 16)
    event_sequences: Optional[Tensor]       # (batch, seq_len, feat_dim)
    multidisciplinary_features: Optional[Tensor]  # (batch, 24)
    sample_weights: Optional[Tensor]        # (batch,)
```

12. Appendix: Config File Structure

All system parameters are managed through a split-config scheme. `config_builder.py` is the **single source of truth**: it merges `pipeline.yaml` (infrastructure-agnostic) and `datasets/santander.yaml` (dataset-specific) into the resolved config consumed by all pipeline stages. No pipeline stage reads raw YAML directly.

`pipeline.yaml`: Task definitions, model structure, training parameters, AWS infrastructure settings

- `tasks`: 12 tasks (name, type, loss, loss_weight, label_col)
- `model.ple`: num_layers, extraction_dim, expert_basket
- `model.adatt`: task_groups, intra/inter strength
- `model.logit_transfers`: 3 transfer edges
- `training`: batch_size, epochs, learning_rate, amp
- `aws`: instance_type, spot, budget_limit

`datasets/santander.yaml`: Dataset-specific overrides (column names, split parameters, adapter settings). Adding a new dataset requires only this file — no changes to `pipeline.yaml` or any Python code.

`feature_groups.yaml`: 17 feature group definitions

- `group_type`: transform (existing column transformation) | generate (Generator invocation)
- `generator`: Reference to Pool's Generator name
- `generator_params`: input_filter (dtype, exclude_binary, min_nunique, etc.)
- `target_experts`: List of experts that receive this group's features
- `output_dim`: Output dimension

13. Ops/Audit Agent Layer

Two autonomous diagnostic agents operate as a separate layer, asynchronously monitoring the entire pipeline:

- **OpsAgent**: monitors 7 checkpoints (ingestion through A/B testing), performs cross-checkpoint correlation analysis
- **AuditAgent**: audits 5 viewpoints (fairness, concentration, reason quality, regulatory compliance, data lineage)

48 checklist items are automatically evaluated. WARN/FAIL items undergo 3-agent consensus (Sonnet×3) producing diagnostic results with minority report preservation. Diagnostic history accumulates in a LanceDB-based `DiagnosticCaseStore`.

Fully decoupled from the serving path (3 serving agents) — agent failures never degrade customer-facing service.

13.1. Serverless Scheduling: EventBridge + Lambda

Both OpsAgent and AuditAgent are deployed as **serverless Lambda functions** scheduled by Amazon EventBridge. This eliminates the need for always-on compute for diagnostic workloads:

- **OpsAgent Lambda**: triggered on a configurable cron (e.g., every 6 hours). Reads pipeline checkpoint artifacts from S3, performs correlation analysis, writes findings to DynamoDB.
- **AuditAgent Lambda**: triggered after each SageMaker training job completion event via EventBridge rule. Reads model outputs and feature stats, performs fairness and lineage checks, writes audit records to DynamoDB.

13.2. CloudFormation Monitoring Stack

The monitoring infrastructure is defined as a single CloudFormation stack, ensuring reproducible deployment and version-controlled infrastructure:

Component	Role
EventBridge Rules	Schedule OpsAgent + AuditAgent Lambda; route SageMaker job events
CloudWatch Alarms	Drift threshold breach, Lambda error rate, serving latency P99
SNS Topics	Alert routing: WARN → Slack webhook, FAIL → PagerDuty + email
DynamoDB Tables	PipelineState (job resume state), reason_cache (L2a cache), audit log tables
LanceDB	DiagnosticCaseStore (audit history) + TemporalFactStore + recommendation_cases (single shared instance)
Lambda Functions	OpsAgent, AuditAgent, FallbackRouter health check

CloudWatch metric filters extract per-task validation metrics from SageMaker training logs and expose them as custom metrics. This enables CloudWatch Alarms to fire on task-level degradation without requiring a separate monitoring service.

Detailed design: Design Document 11 (`docs/design/11_ops_audit_agent.typ`)

13.3. On-Premises (Air-Gapped) Deployment

In air-gapped on-premises environments, the same pipeline runs on local GPU (RTX 4070), using Exaone 3.5 7.8B (reason generation) and Qwen 2.5 14B Q4 (agent consensus) instead of Bedrock. Data never leaves the premises, providing structural data protection.

When switching domains, replacing only these 2 files configures an entirely different recommendation system without any code changes.